CSE 144 Applied Machine Learning

Final Project

UC Santa Cruz

Due: June 10, 2026, 11:59 PM

1 Transfer Learning Challenge

In the previous assignment, you have seen how to train a Simple CNN on the CIFAR-10 dataset, which does not require sophisticated data preprocessing or model design choices to achieve decent accuracy. In this section, your task is to apply transfer learning techniques learned in class to train a classifier for a more difficult dataset.

Please use the URL below to access the Kaggle page of this competition:

`https://www.kaggle.com/competitions/ucsc-cse-144-spring-2026-final-project`

The Kaggle page is for evaluation and submission only. Please refer to these instructions for details.

1.1 Dataset

To save computation, we sample from different datasets, forming a total of 100 classes. For each class, there are 10 sampled training images and 10 evaluation images. All images in the training and evaluation sets are colored, but they may have different shapes. Thus, it is strongly advised to resize all images to the same size (e.g., 224×224) before training.

The dataset directory is structured as follows:

```
train/
  - 0/
    - 0.jpg
    - 1.jpg
    ...
    - 9.jpg
```

```
  - 1/
    ...
  - 99/
    ...

test/
  - 0.jpg
  - 1.jpg
  ...
  - 999.jpg
  - sample_submission.csv
```

In the `train/` directory, there are 100 directories, each named by a string label. Each directory contains approximately 10 images of a particular class. The `test/` directory contains 1000 unlabeled images.

You may only use the images in the `train/` directory for training. After finishing hyperparameter tuning, you will load all test images and predict their labels using your final model.

You must submit a `submission.csv` file with two columns: {ID, Label}. A sample file `sample_submission.csv` is provided as a template.

You should leave the ID column unchanged and fill in the Label column with your predictions.

The training samples are already grouped by categories. Note that different implementations might lead to different category orders. For example, class "0" may be mapped to class 1. To ensure correct evaluation, you must use:

- Label 0 for class "0"
- Label 1 for class "1"
- And so on

If the label order is scrambled, the best possible result will be no better than random guessing.

1.2 Implementation and Training

We strongly suggest reusing code from the previous assignment. You may modify the existing code by adopting:

- More advanced architectures
- More sophisticated data augmentations
- Tuned hyperparameters

Unlike previous assignments, you are required to load strong pre-trained model weights and fine-tune them on this dataset. This enables fast convergence and decent accuracy with limited data and computation.

See TorchVision models for available pretrained weights.

Advice: Modern models are often several orders of magnitude larger than models used previously. These introduce significant computational overhead.

Before using large models, try training on a small subset of the data to check:

- Whether the model fits in GPU memory
- How long training takes

Since the dataset contains only 1000 samples, very large models may overfit.

1.3 Reproducibility

Please ensure that your final training and testing accuracies are reproducible. With your submitted code and documents, others should be able to reproduce your results with similar means and standard deviations.

Good reproducibility practices include:

- Using a fixed random seed
- Reporting averaged metrics overall multiple runs
- Providing detailed execution instructions

If your final accuracy cannot be reproduced using your submitted code, your competition score may be affected.

1.4 Submission

As described in Section 1.1, you must submit a CSV file with image IDs and predicted labels on Kaggle.

The public leaderboard provides an estimated score based on approximately 10% of the final test set. Do not use it to infer final performance.

Use the public leaderboard only to verify submission format.

Additionally, you must submit a **public** Github repository link to Canvas:

1. The public GitHub repository must include all source code, a project report in PDF format, and a Google drive link to the trained model weights.
2. The repository README should clearly document how to run both training and inference using the provided model.
3. It must also include a screenshot showing your team's position on the Kaggle leaderboard, with the screenshot referenced directly in the README.
4. The report with instructions, experimental setup, etc. Please refer to the "sample report".

1.5 Grading Criteria

The project score is divided into three parts (maximum total score: 100).

1. **Kaggle test set accuracy.** You need to pass the baseline test accuracy of **60%**. Let x be your submission accuracy (in %). Define the Kaggle score:

$$S_{\text{kaggle}} = \begin{cases} 0, & \text{if no submission,} \\ 70 + \max(0, x - 60), & \text{otherwise,} \end{cases}$$

   where the bonus term $\max(0, x - 60)$ has no upper limit.

2. **Presentation.** Let $p \in [0, 10]$ be the score of the presentation.

$$S_{\text{pres}} = \begin{cases} p, & \text{if there's a presentation,} \\ -10, & \text{if no presentation.} \end{cases}$$

3. **Report + code + model weights.** Let $r \in [0, 10]$ be the graded score based on the provided report and uploaded code/model weights.

$$S_{\text{repo}} = \begin{cases} r, & \text{if report+code+weights are submitted,} \\ -10, & \text{otherwise.} \end{cases}$$

The total project score is $S_{\text{total}} = \min\big(100,\ S_{\text{kaggle}} + S_{\text{pres}} + S_{\text{repo}}\big)$.